

SWING SENSE

Plano de Gerenciamento de Riscos de Segurança da Informação

Relatório de Vulnerabilidades do Sistema e do Código — Redes de Computadores e Cibersegurança

Projeto: Swing Sense — rede social de treinos de tênis com raquete inteligente (ESP32)

Data de criação/atualização: 20/09/2026

Versão: 1.0

Componentes analisados: Backend (Dart/shelf), App (Flutter), Banco (PostgreSQL/Supabase)

Integrantes do grupo:

Antônio Petri • Gabriel Higobassi Paschoal

Lucas Silva Maciel • Thiago Akira Higa Mitami

Documento elaborado a partir de revisão de código (SAST manual) e testes dinâmicos (DAST) executados em ambiente controlado.

Plano de Gerenciamento de Riscos

Projeto: Swing Sense | Data: 20/09/2026 | Versão: 1.0

1. Introdução

Este documento descreve como o gerenciamento de riscos de **segurança da informação** será estruturado e realizado para o projeto **Swing Sense**, uma rede social de treinos de tênis integrada a uma raquete inteligente (ESP32). O objetivo é identificar, analisar, planejar respostas e monitorar os riscos (vulnerabilidades) do sistema e do código, aumentando a probabilidade de sucesso do projeto e minimizando o impacto de eventos negativos — em especial o comprometimento de contas e a exposição de dados de usuários.

O escopo cobre a API em Dart (`swing_sense_backend`), o aplicativo Flutter (`swing_sense_frontend`) e o banco PostgreSQL hospedado no Supabase. As vulnerabilidades aqui registradas foram **confirmadas empiricamente** em um ambiente de teste isolado (cópia do backend contra um PostgreSQL local com o mesmo schema), sem qualquer alteração no código-fonte do projeto.

2. Metodologia

A abordagem para o gerenciamento de riscos neste projeto será a seguinte:

- Identificação de Riscos:** revisão manual de código-fonte (análise estática/SAST) de todas as rotas, middleware, utilitários e schema; testes dinâmicos (DAST) com requisições reais à API; e mapeamento contra o **OWASP Top 10 (2021)**.
- Análise de Riscos:** cada risco é analisado qualitativamente quanto a **probabilidade e impacto**, resultando em um nível segundo a matriz da Seção 7. Riscos de alto nível recebem prova de conceito documentada.
- Planejamento de Respostas:** para cada risco significativo definimos a estratégia (*prevenir, transferir, mitigar ou aceitar*) e a ação técnica correspondente (a "segurança a aplicar").
- Monitoramento e Controle:** os riscos são monitorados ao longo do projeto; o Registro de Riscos (Seção 11) é um documento vivo, atualizado a cada nova versão do sistema, com reavaliação da eficácia das respostas.

3. Papéis e Responsabilidades

Papel	Responsabilidades no Gerenciamento de Riscos
Gerente de Projeto / Coordenação de Segurança Lucas Silva Maciel	Liderar o processo de gerenciamento de riscos, garantir que o plano seja seguido, priorizar as correções e comunicar o status dos riscos ao grupo e ao orientador.
Equipe do Projeto (Desenvolvimento e Testes) Antônio Petri, Gabriel Higobassi Paschoal, Thiago Akira Higa Mitami	Participar ativamente da identificação, análise e monitoramento dos riscos; executar os testes de segurança; implementar as respostas/correções designadas no código.

Patrocinador / Orientador Docente responsável pela disciplina	Fornecer diretrizes e recursos; tomar decisões sobre riscos que excedam a autoridade da equipe; validar o plano.
Partes Interessadas Usuários finais / atletas	Fornecer informações para a identificação de riscos (expectativas de privacidade e integridade dos dados) e estar cientes dos principais riscos do projeto.

4. Orçamento e Cronograma para Gerenciamento de Riscos

Orçamento: por se tratar de projeto acadêmico, o "orçamento" é medido em **horas de trabalho** da equipe. Estima-se uma reserva de contingência de ~15% do esforço de desenvolvimento dedicada a atividades de segurança (identificação, correção e reteste das vulnerabilidades).

Cronograma:

- **Identificação inicial de riscos:** concluída nesta Entrega 1 (revisão de código + testes dinâmicos).
- **Reuniões de acompanhamento de riscos:** a cada sprint (quinzenal), junto às revisões de código.
- **Revisões do Plano de Gerenciamento de Riscos:** a cada marco/entrega do projeto, com reteste dos riscos corrigidos.

5. Categorias de Riscos

Os riscos são categorizados para facilitar a identificação e análise:

- **Técnicos:** relacionados a autenticação, controle de acesso, criptografia, configuração e validação de entrada (a maioria das vulnerabilidades encontradas — ex.: R-01, R-02, R-05).
- **Gerenciais:** relacionados a planejamento, comunicação e controle das correções de segurança (ex.: priorização e reteste das falhas).
- **Organizacionais:** relacionados a processos e ambiente do projeto — ex.: configuração de infraestrutura/containers (R-10).
- **Externos:** relacionados a fornecedores e dependências — ex.: gestão de credenciais do Supabase (R-12).

6. Definições de Probabilidade e Impacto

Probabilidade

Nível	Descrição	Valor
Muito Alta	Quase certo de ocorrer / explorável trivialmente	> 80%
Alta	Provável de ocorrer	61% – 80%
Média	Pode ocorrer	41% – 60%
Baixa	Improvável de ocorrer	21% – 40%
Muito Baixa	Muito improvável de ocorrer	< 20%

Impacto

Adaptado ao contexto de segurança: o impacto é medido pelo dano à confidencialidade, integridade e disponibilidade dos dados e do serviço.

Nível	Descrição (segurança da informação)
Muito Alto	Comprometimento total: acesso a qualquer conta / vazamento massivo de dados
Alto	Comprometimento de contas individuais, interceptação de credenciais ou indisponibilidade relevante
Médio	Acesso/alteração indevida de dados específicos ou abuso funcional significativo
Baixo	Exposição de informação limitada ou abuso de baixo alcance
Muito Baixo	Impacto mínimo, sem dano direto a dados ou usuários

7. Matriz de Probabilidade e Impacto

Esta matriz é utilizada para priorizar os riscos. As células indicam o nível resultante e, quando aplicável, os riscos posicionados naquela combinação.

Prob. ↓ / Impacto →	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	Médio	Alto	Alto	Extremo	Extremo
Alta	Baixo	Médio R-04	Alto R-05	Alto R-02	Extremo R-01
Média	Baixo	Médio R-07, R-11	Médio R-03, R-06, R-08	Alto R-09	Alto
Baixa	Muito Baixo	Baixo	Baixo R-10	Médio	Médio R-12
Muito Baixa	Muito Baixo	Muito Baixo	Baixo	Baixo	Médio

Extremo/Alto: requerem atenção imediata e plano de resposta detalhado. **Médio:** devem ser monitorados de perto.
Baixo/Muito Baixo: monitorados como rotina do projeto.

8. Tolerância ao Risco das Partes Interessadas

A equipe e o orientador adotam **baixa tolerância a riscos que afetem a segurança e a privacidade dos dados dos usuários** (autenticação, controle de acesso e exposição de dados pessoais) — riscos de nível *Extremo* e *Alto* devem ser corrigidos antes da próxima entrega. Há tolerância moderadamente maior para riscos de configuração/infraestrutura de baixo impacto (ex.: R-07, R-10), que podem ser tratados em etapas seguintes, desde que monitorados.

9. Formatos de Relatórios

- **Registro de Riscos:** mantido neste documento (Seção 11) e em planilha compartilhada, com os campos: ID, vulnerabilidade, categoria, referência OWASP, localização, probabilidade, impacto, nível, resposta e ação.

- **Relatório de Status dos Riscos:** incluído nos relatórios de progresso de cada entrega, apresentando os principais riscos, o status das correções e novos riscos identificados.

10. Segurança já Aplicada no Sistema

Em atendimento ao requisito "sistema com segurança aplicada", registram-se os controles de segurança **já presentes** no Swing Sense, confirmados na análise. Eles reduzem a probabilidade/impacto de várias ameaças e servem de base para as melhorias propostas.

Controle de segurança aplicado	Evidência / localização
Hash de senha com bcrypt (salt por senha)	lib/src/utls/password_util.dart — senhas nunca armazenadas em texto claro.
Consultas parametrizadas (SQL Injection mitigada)	postgres_store.dart usa Sql.named com parâmetros; teste com payload "' OR '1'='1" retornou HTTP 401 sem bypass.
Autenticação por JWT nas rotas protegidas	middleware/auth_middleware.dart exige Bearer válido; rotas sob /users,/feed,/sessions,/goals,/devices.
.env fora do versionamento	.gitignore inclui .env — credenciais não vão para o repositório (base para R-12).
Verificação de posse na maioria das rotas	GET /sessions/:id, updateSession, metas e devices já filtram por user_id.
Mensagem de login genérica	POST /auth/login não revela se o e-mail existe ('Email ou senha incorretos').
SSL na conexão com o banco	database.dart / DB_SSL_MODE=require — tráfego API↔Supabase criptografado.

Injeção de SQL — teste de validação: o envio do payload clássico ' OR '1'='1 no login retornou **HTTP 401** (falha segura), sem bypass e sem erro de banco, confirmando o uso correto de consultas parametrizadas.

11. Registro de Riscos (Relatório de Vulnerabilidades)

Resumo priorizado dos riscos identificados e testados. O detalhamento com evidência de cada item vem na sequência.

ID	Vulnerabilidade	Categoria	Prob.	Impacto	Nível	Resposta
R-01	Segredo JWT padrão/hardcoded no código	Técnico	Alta	Muito Alto	Extremo	Prevenir/Mitigar
R-02	IDOR na ingestão de telemetria (POST /sessions/:id/events)	Técnico	Alta	Alto	Extremo	Prevenir
R-03	IDOR em curtidas e comentários (/feed/:id/like, /comments)	Técnico	Média	Médio	Médio	Mitigar
R-04	Enumeração de usuários no cadastro	Técnico	Alta	Baixo	Médio	Mitigar
R-05	Ausência de proteção contra força bruta (login)	Técnico	Alta	Médio	Alto	Mitigar
R-06	Política de senha fraca	Técnico	Média	Médio	Médio	Mitigar

R-07	CORS permissivo (Access-Control-Allow-Origin: *)	Técnico	Média	Baixo	Baixo	Mitigar
R-08	Tokens JWT de longa duração sem revogação	Técnico	Média	Médio	Médio	Mitigar
R-09	Transporte sem TLS (HTTP em texto claro)	Técnico	Média	Alto	Alto	Mitigar
R-10	Container executando como root em modo de desenvolvimento	Organizacional	Baixa	Médio	Baixo	Mitigar
R-11	Falta de validação/limite de payload (abuso de armazenamento/DoS)	Técnico	Média	Baixo	Baixo	Mitigar
R-12	Exposição de credenciais do banco (Supabase) por vazamento de segredo	Externo	Baixa	Muito Alto	Alto	Transferir/Mitigar

Detalhamento e evidências

R-01 — Segredo JWT padrão/hardcoded no código **Extremo**

Categoria: Técnico • **Referência:** A02:2021 – Falhas Criptográficas / A07 – Falhas de Identificação e Autenticação

Probabilidade: Alta • **Impacto:** Muito Alto • **Nível de risco:** Extremo

Descrição: Se a variável JWT_SECRET não for definida, a API assina os tokens com um segredo fixo conhecido, presente no repositório. Como o algoritmo é HS256 (chave simétrica), qualquer pessoa que conheça o segredo forja tokens válidos para qualquer usuário, sem precisar da senha.

Localização no código: lib/src/app.dart (linha 38): `env('JWT_SECRET') ?? 'dev-secret-troque-em-producao';` e `.env.example` com `JWT_SECRET=troque-este-segredo...`

Evidência (teste real na API):

```
# Token forjado apenas com o segredo padrão do código (sem a senha da vítima)
$ python3 forge.py -> role=admin, sub=<id-da-vitima>
$ curl /users/me -H "Authorization: Bearer <TOKEN_FORJADO>"
{
  "id": "f459625d-8205-4c7d-ae6e-6056a73cf25d",
  "email": "vitima@swingsense.app",  <- conta da vítima acessada
  "level": "avancado", ...
}
```

Estratégia de resposta: Prevenir/Mitigar

Ação de mitigação / segurança a aplicar: Gerar JWT_SECRET aleatório forte (≥ 256 bits) por ambiente, obrigatório na inicialização (falhar se ausente); remover o valor padrão do código e o placeholder reutilizável do `.env.example`; considerar rotação de chave e algoritmo assimétrico (RS256). Nunca versionar segredos.

R-02 — IDOR na ingestão de telemetria (POST /sessions/:id/events) **Extremo**

Categoria: Técnico • **Referência:** A01:2021 – Quebra de Controle de Acesso (IDOR)

Probabilidade: Alta • **Impacto:** Alto • **Nível de risco:** Extremo

Descrição: Diferente de outras rotas de sessão, o endpoint de eventos não confere a posse da sessão. Qualquer usuário autenticado que conheça/adivinhe o ID de uma sessão injeta telemetria falsa (velocidade, rotação, ângulo) na sessão de outro atleta, corrompendo estatísticas e o feed social.

Localização no código: `lib/src/routes/session_routes.dart` (linhas 68–88) e `postgres_store.insertSessionEvents` – inserem eventos sem verificar se a sessão pertence ao usuário autenticado.

Evidência (teste real na API):

```
# Atacante (token próprio) grava evento na sessão da VÍTIMA
$ curl -X POST /sessions/<ID_SESSAO_VITIMA>/events \
  -H "Authorization: Bearer <TOKEN_ATACANTE>" \
  -d '{"events":[{"eventType":"forehand","ballSpeedKmh":999,...}]}'
HTTP 201 {"inserted":1}
# Confirmação no banco:
event_type | ball_speed_kmh | dono_da_sessao
forehand   | 999.0                  | vitima@swingsense.app
```

Estratégia de resposta: Prevenir

Ação de mitigação / segurança a aplicar: Validar posse antes de inserir: carregar a sessão e exigir `session.user_id == request.userId` (mesmo padrão já usado em GET /sessions/:id/events e updateSession), retornando 404/403 caso contrário.

R-03 — IDOR em curtidas e comentários (/feed/:id/like, /comments) **Médio**

Categoria: Técnico • **Referência:** A01:2021 – Quebra de Controle de Acesso

Probabilidade: Média • **Impacto:** Médio • **Nível de risco:** Médio

Descrição: É possível curtir e comentar qualquer sessão pelo ID, mesmo de usuários que o atacante não segue e cujas sessões não apareceriam no feed dele, permitindo abuso/spam direcionado.

Localização no código: `lib/src/routes/feed_routes.dart` (linhas 19–50) – não verifica se a sessão é visível ao usuário (própria ou de quem ele segue).

Evidência (teste real na API):

```
$ curl -X POST /feed/<ID_SESSAO_VITIMA>/comments \
  -H "Authorization: Bearer <TOKEN_ATACANTE>" \
  -d '{"content":"invadido pelo atacante"}'
HTTP 201 {"content":"invadido pelo atacante", ...}
$ curl -X POST /feed/<ID_SESSAO_VITIMA>/like ... -> HTTP 200
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Restringir like/comentário a sessões visíveis ao usuário (próprias ou de perfis seguidos), validando a relação antes de gravar.

R-04 — Enumeração de usuários no cadastro Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Probabilidade: Alta • **Impacto:** Baixo • **Nível de risco:** Médio

Descrição: A resposta distingue e-mails cadastrados (409) de novos (201), permitindo a um atacante mapear quais e-mails possuem conta — insumo para phishing e ataques de senha.

Localização no código: lib/src/routes/auth_routes.dart (linhas 26–29) — retorna HTTP 409 'Já existe uma conta com este email'.

Evidência (teste real na API):

```
e-mail já cadastrado : HTTP 409 {"error":"Ja existe uma conta com este email"}
e-mail inexistente  : HTTP 201 {"token": "..."} 
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Padronizar resposta genérica no cadastro (ex.: fluxo de confirmação por e-mail) e aplicar rate limiting; o login já usa mensagem genérica — replicar o mesmo cuidado.

R-05 — Ausência de proteção contra força bruta (login) Alto

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Probabilidade: Alta • **Impacto:** Médio • **Nível de risco:** Alto

Descrição: Não há limite de tentativas de login. Combinado com política de senha fraca (R-06), viabiliza ataques de força bruta e credential stuffing.

Localização no código: lib/src/routes/auth_routes.dart (POST /login) — sem rate limiting, CAPTCHA ou bloqueio temporário.

Evidência (teste real na API):

```
# 20 tentativas de login em sequência
20/20 requisições processadas – todas HTTP 401,
nenhuma HTTP 429 (Too Many Requests), sem atraso/bloqueio. 
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Implementar rate limiting por IP/conta, backoff progressivo e bloqueio temporário após N falhas; considerar 2FA e monitoramento de tentativas.

R-06 — Política de senha fraca Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Probabilidade: Média • **Impacto:** Médio • **Nível de risco:** Médio

Descrição: Senhas triviais como '123456' são aceitas, reduzindo o custo de ataques de dicionário.

Localização no código: `lib/src/routes/auth_routes.dart` (linha 22) – exige apenas 6 caracteres, sem complexidade.

Evidência (teste real na API):

```
$ curl -X POST /auth/register -d '{"...","password":"123456"}'  
HTTP 201 (conta criada com senha trivial)
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Exigir mínimo de 8-12 caracteres, verificar contra listas de senhas vazadas (ex.: HavelBeenPwned k-anonymity) e orientar o uso de senhas fortes.

R-07 — CORS permissivo (Access-Control-Allow-Origin: *) Baixo

Categoria: Técnico • **Referência:** A05:2021 – Configuração Incorreta de Segurança

Probabilidade: Média • **Impacto:** Baixo • **Nível de risco:** Baixo

Descrição: Qualquer site pode chamar a API a partir do navegador. Como a autenticação usa Bearer token (não cookies), o risco imediato é menor, mas amplia a superfície de ataque e facilita abuso caso tokens sejam expostos no front web.

Localização no código: `lib/src/app.dart` (linhas 18-34) – libera qualquer origem para todos os métodos.

Evidência (teste real na API):

```
$ curl -D - /health -H "Origin: https://site-malicioso.com"  
access-control-allow-origin: *  
access-control-allow-methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Restringir a origem a uma allowlist dos domínios oficiais do app web; evitar '*' em produção.

R-08 — Tokens JWT de longa duração sem revogação Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Probabilidade: Média • **Impacto:** Médio • **Nível de risco:** Médio

Descrição: Tokens válidos por 7 dias e sem mecanismo de revogação: um token vazado permanece utilizável por longo período, mesmo após troca de senha.

Localização no código: `lib/src/app.dart` (linha 39) e `.env.example` — `JWT_EXPIRES_IN_HOURS=168` (7 dias); sem refresh/blacklist.

Evidência (teste real na API):

```
PAYLOAD do token: exp - iat = 604800 s = 168 h (7 dias).  
Não há endpoint de logout no servidor nem lista de revogação.
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Reduzir expiração do access token (ex.: 15-60 min) com refresh token rotativo; manter lista de revogação/versionamento de token para invalidar após logout ou troca de senha.

R-09 — Transporte sem TLS (HTTP em texto claro) Alto

Categoria: Técnico • **Referência:** A02:2021 – Falhas Criptográficas

Probabilidade: Média • **Impacto:** Alto • **Nível de risco:** Alto

Descrição: Comunicação app↔API por HTTP permite interceptação de credenciais e tokens em redes não confiáveis (Wi-Fi público). A conexão com o banco (Supabase) já usa SSL (DB_SSL_MODE=require) — o gargalo é o tráfego cliente↔API.

Localização no código: `frontend lib/core/config/app_config.dart` (`http://...:8080`) e `Dockerfile/compose` expõem HTTP puro; a API não termina TLS.

Evidência (teste real na API):

```
AppConfig.apiUrl -> "http://10.0.2.2:8080" / "http://localhost:8080"  
Sem proxy TLS no docker-compose; API escuta HTTP puro em :8080.
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Servir a API atrás de proxy/HTTPS (TLS) em produção, forçar HSTS, usar `https://` no app e habilitar restrição de cleartext no Android/iOS.

R-10 — Container executando como root em modo de desenvolvimento Baixo

Categoria: Organizacional • **Referência:** A05:2021 – Configuração Incorreta de Segurança

Probabilidade: Baixa • **Impacto:** Médio • **Nível de risco:** Baixo

Descrição: Rodar como root e em modo interpretado aumenta a superfície e o impacto de uma eventual exploração de container, além de reduzir desempenho e previsibilidade em produção.

Localização no código: Dockerfile – usa 'dart run bin/server.dart' (JIT/dev) como root, sem usuário não privilegiado nem build AOT.

Evidência (teste real na API):

```
Dockerfile: CMD ["dart","run","bin/server.dart"] (sem USER não-root; sem 'dart compile exe').
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Compilar com 'dart compile exe', criar usuário não-root (USER app), imagem mínima e healthcheck; aplicar limites de recurso.

R-11 — Falta de validação/limite de payload (abuso de armazenamento/DoS) Baixo

Categoria: Técnico • **Referência:** A04:2021 – Design Inseguro

Probabilidade: Média • **Impacto:** Baixo • **Nível de risco:** Baixo

Descrição: Listas de eventos e textos (comentários, notas) não têm limite. Um usuário pode enviar cargas enormes, consumindo banco e memória (negação de serviço/abuso de custo no Supabase).

Localização no código: session_routes (events sem limite de tamanho/quantidade) e feed comments/notes sem limite de comprimento.

Evidência (teste real na API):

```
POST /sessions/:id/events aceita 'events' de tamanho arbitrário;  
comentários/notas sem limite de caracteres (apenas 'não vazio').
```

Estratégia de resposta: Mitigar

Ação de mitigação / segurança a aplicar: Impor limites de tamanho de corpo, quantidade de eventos por requisição e comprimento de textos; validar tipos/faixas dos valores de telemetria.

R-12 — Exposição de credenciais do banco (Supabase) por vazamento de segredo

Alto

Categoria: Externo • **Referência:** A02/A05 – Gestão de Segredos

Probabilidade: Baixa • **Impacto:** Muito Alto • **Nível de risco:** Alto

Descrição: Caso um .env seja acidentalmente commitado, logado ou exposto, um terceiro obtém acesso direto ao PostgreSQL do Supabase (dados de todos os usuários). Risco de fornecedor/externo combinado a erro operacional.

Localização no código: Credenciais em .env (DB_PASSWORD, JWT_SECRET). .env está no .gitignore (controle já aplicado), mas segredos trafegam em texto no arquivo/ambiente.

Evidência (teste real na API):

Controle já presente: .gitignore contém '.env' (não versionado).
Resíduo: sem cofre de segredos; segredos em texto no ambiente/arquivo.

Estratégia de resposta: Transferir/Mitigar

Ação de mitigação / segurança a aplicar: Usar cofre de segredos (Supabase Vault, Doppler, GitHub Actions Secrets), rotação periódica de senha/JWT, princípio do menor privilégio no usuário de banco e verificação de segredos no CI (git-secrets/gitleaks).

12. Acompanhamento

O processo de gerenciamento de riscos será revisado em cada marco importante do projeto para garantir sua eficácia e incorporar as lições aprendidas. O Registro de Riscos é um **documento vivo** e será atualizado continuamente: a cada correção, o risco correspondente será **retestado** e seu status atualizado (aberto → em correção → mitigado/fechado). As lições aprendidas — sobretudo a importância de controle de acesso por posse (IDOR) e de gestão de segredos — serão aplicadas às próximas funcionalidades, inclusive à futura integração com o hardware real (ESP32).

Aprovação

Nome	Cargo	Assinatura	Data
Lucas Silva Maciel	Gerente do Projeto / Coord. de Segurança	<i>Lucas S. Maciel</i>	20/09/2026
Antônio Petri	Equipe do Projeto	<i>Antônio Petri</i>	20/09/2026
Gabriel Higobassi Paschoal	Equipe do Projeto	<i>Gabriel H. Paschoal</i>	20/09/2026
Thiago Akira Higa Mitami	Equipe do Projeto	<i>Thiago A. H. Mitami</i>	20/09/2026

Anexo — Ambiente de teste: cópia isolada do backend (Dart 3.5.4, postgres 3.5.7) executada contra PostgreSQL 18 local com o schema oficial (lib/src/db/schema.sql). Nenhum arquivo do projeto foi modificado. Código-fonte analisado disponível em SRC/Swing Sense/ no repositório do projeto.